

MLOps

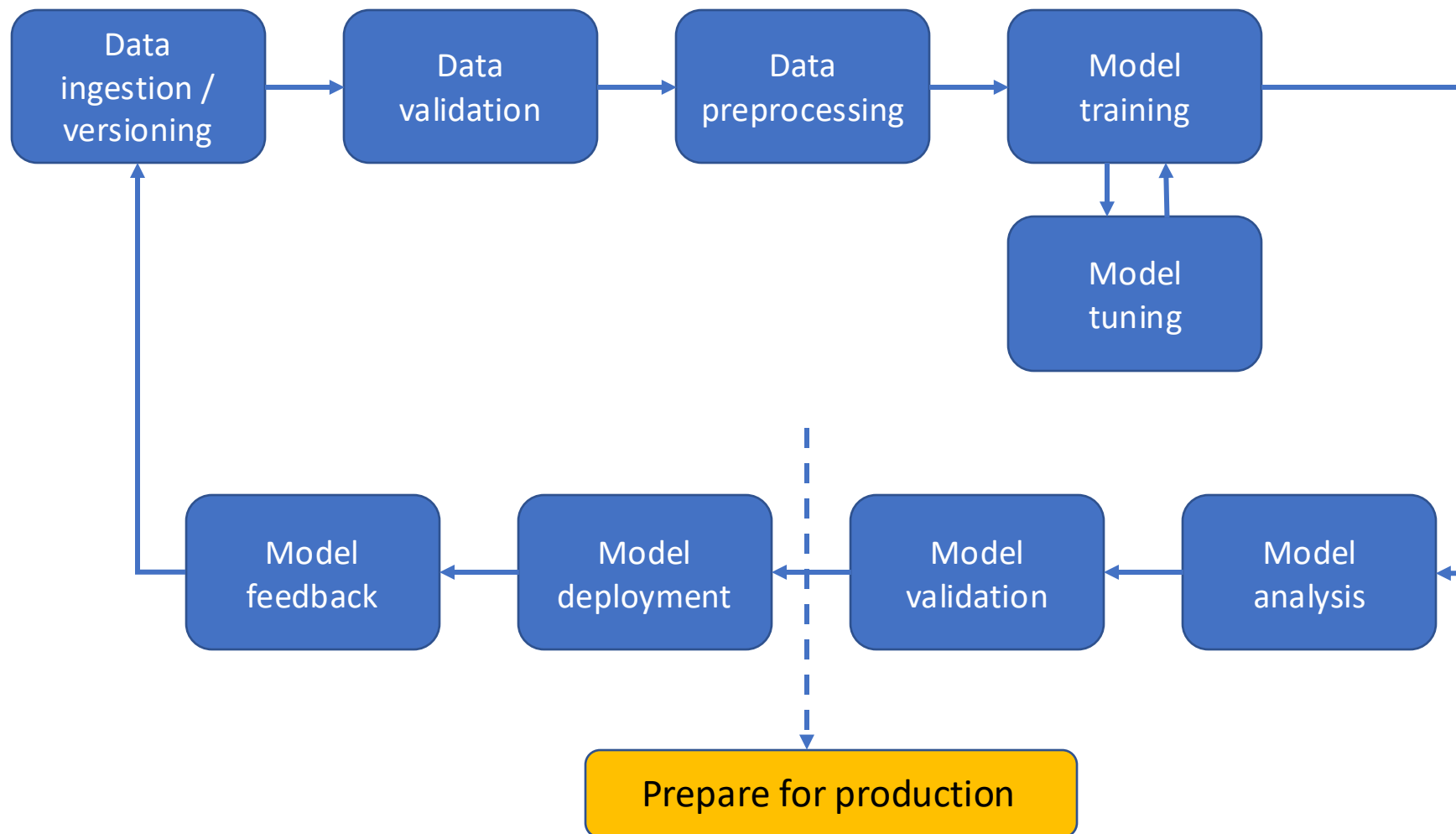
Preparing for Production – Deployment - Scaling

Session 5

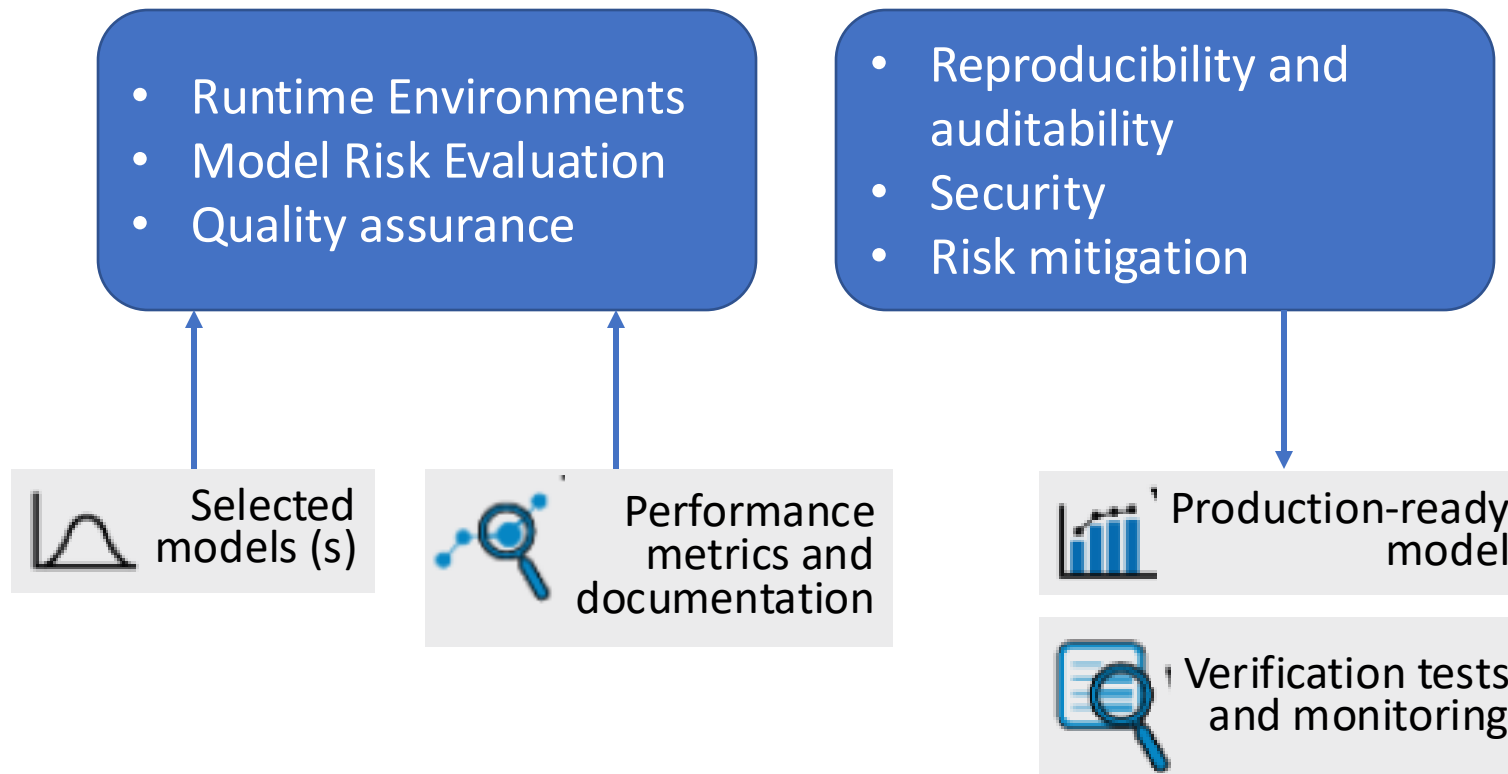
Juan Ramírez
jramirezg@faculty.ie.edu

Preparing for Production

Preparing for Production



Preparing for Production



Runtime Environments

- First step is making sure it's technically possible
- Production environments: custom-built services, data science platforms, dedicated services like TensorFlow Serving, low-level infrastructure like Kubernetes clusters, JVMs on embedded systems, etc.
- In some organizations multiple heterogeneous environments coexist

Ideally: models running in the development environment would be validated and sent 'as is' to production

Adaptation from Development to Production Environments

Hours, days, weeks, ...

The development and production platforms are from the same vendor or are interoperable

Model reimplemented from scratch –possible by another team and in another programming language

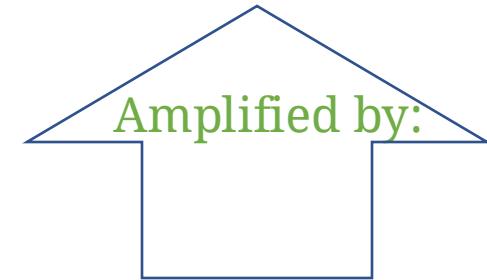
Reasons:

- **Tooling**: consider the format needed in production early. Ex. Model developed using Scikit-Learn and production is Java based environment that expects PMML or ONNX
- **Performance**:
 - Python model will typically have higher latency for scoring a single record than its equivalent converted to C++
 - Production model must run on a low-power device
- **Data access**: always take into account the data needed to score the model (data frozen and bundled with the model, access to data storage,...)

Model Risk Evaluation

- Back to concerns:
 - What if the model acts in the worst imaginable way?
 - What if a user manages to extract the training data or the internal logic of the model?
 - What are the financial, business, legal, safety, and reputational risks?

- Broad use of the model
- A rapidly changing environment
- Complex interactions between models



ML model risk
originates
essentially from:

- Bugs, errors in designing, training, or evaluating the model (including data prep)
- Bugs in the runtime framework, bugs in the model post-processing/conversion, or hidden incompatibilities between the model and its runtime
- Low quality of training data
- High difference between production data and training data
- Expected error rates, but with failures that have higher consequences than expected
- Misuse of the model or misinterpretation of its outputs
- Adversarial attacks
- Legal risk originating in particular from liability for the model output
- Reputational risk due to bias, unethical use of machine learning, etc.

IBM AI Risk Atlas

- [AI risk atlas — Docs | IBM watsonx](#)

- Risks associated with input
 - Training and tuning phase
 - Inference phase
- Risks associated with output
- Non-technical Risks

AI risk atlas

Last updated: May 07, 2024

Explore this atlas to understand some of the risks of working with generative AI, foundation models, and machine learning models.

Risks are categorized with one of these tags:

- Traditional AI risks (applies to traditional models as well as generative AI)
- Risks amplified by generative AI (might also apply to traditional models)
- New risks specifically associated with generative AI

Risks associated with input



Training and tuning phase



Fairness

[Data bias](#) Amplified



Robustness

[Data poisoning](#) Traditional



Value alignment

[Data curation](#) Amplified
[Downstream retraining](#) New



Data laws

[Data transfer](#) Traditional



Intellectual property

[Data usage rights](#) Amplified



Transparency

[Data transparency](#) Amplified

Quality Assurance

- QA should accompany all stages
- **Testing**: applying the model to carefully curated data and validating measurements against requirements
 - Generate data that cover certain number of scenarios (not only 'real world' data) and split data into subpopulations (evaluate fairness)
 - Collect metrics: statistical (accuracy, precision, recall, etc.) as well as computational (average latency, 95th latency percentile, etc.) -> set up thresholds
 - Compare the model with its previous version: 'Champion/Challenger' approach or checking that a metric does not suddenly drop
 - Check for model stability changing individual features slightly

Reproducibility and Auditability



- **Reproducibility** in MLOps also involves the ability to easily rerun the exact same experiment:
 - Detailed documentation
 - The data used for training and testing
 - An artifact that bundles the implementation of the model plus the full specification of the environment it was run
- **Auditability**: access the full history of the ML pipeline from a central and reliable storage and to easily fetch metadata on all model versions including:
 - The full documentation
 - An artifact that allows running the model with its exact initial environment
 - Test results, including model explanations and fairness reports
 - Detailed model logs and monitoring metadata

Security and Risk Mitigation

- After risk identification you need to set up Controls

Information Security Controls

- Process controls
- Information security
- Compliance management
- Personal data protection controls
- Data backup
- Data destruction
- Software development controls
- Cloud service controls
- Outsourcing controls
- Incident registration and treatment
- Audit plans and review

AI Specific Risk Controls

- Improvement of the quality and safety of AI systems (drift detection, retraining, etc.)
- Controls for technological attacks against AI systems (prompt injection, securitizing endpoints, etc.)
- Security of training datasets (data governance, encryption, etc.)

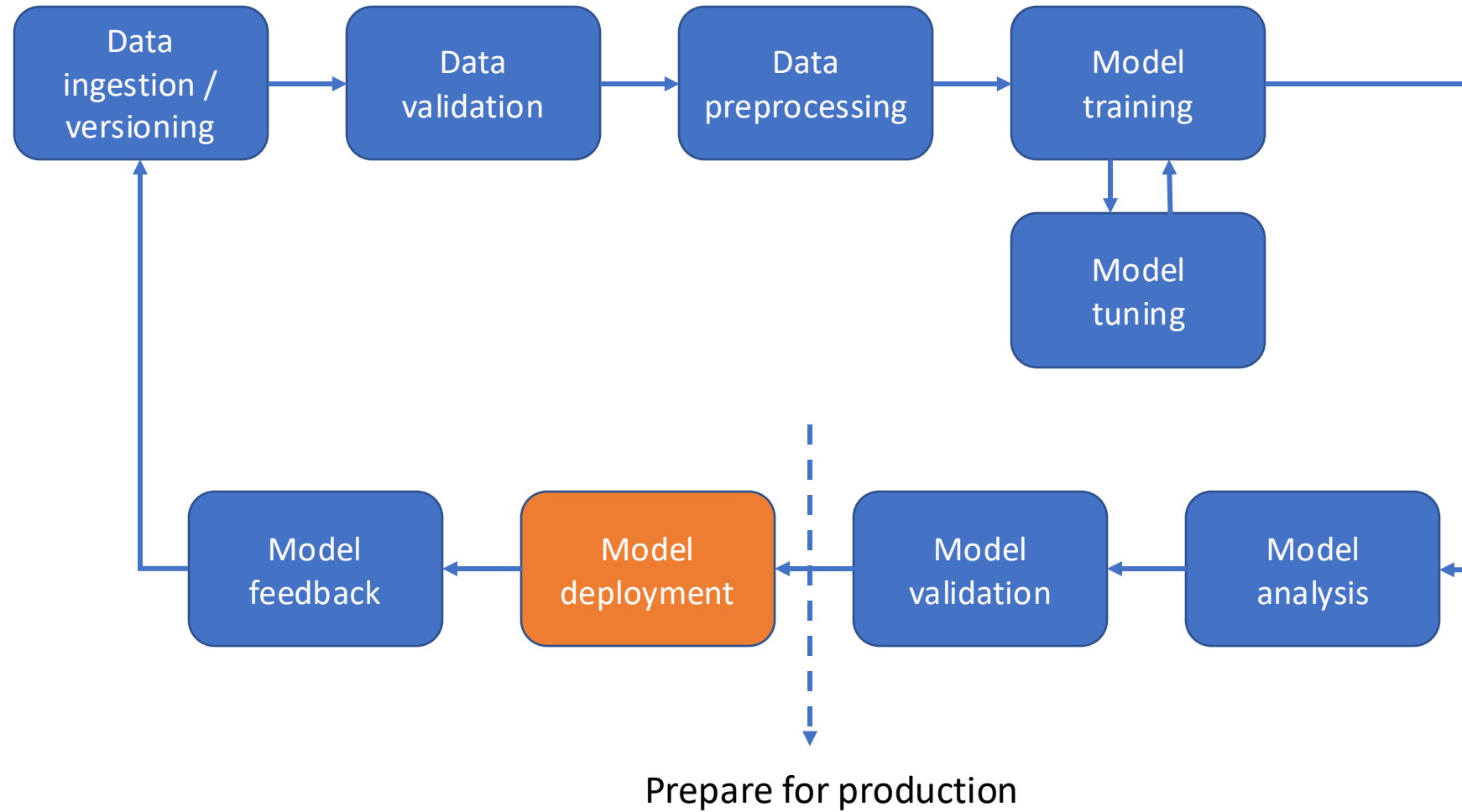
AI System Level Controls

- AI systems registration
- Risk assessments
- Data governance (data lineage, data catalogue, etc.)
- Model monitoring
- Bias
- Accountability

Environments

Model Deployment

Model Deployment



ML Artifact



ML Artifact

- Code for the model and its preprocessing
- Hyperparameters and configuration
- Training and validation data
- Trained model in its runnable form
- An environment including libraries with specific versions, environment variables, etc.
- Documentation
- Code and data for testing scenarios



To long term storage



To test pipeline



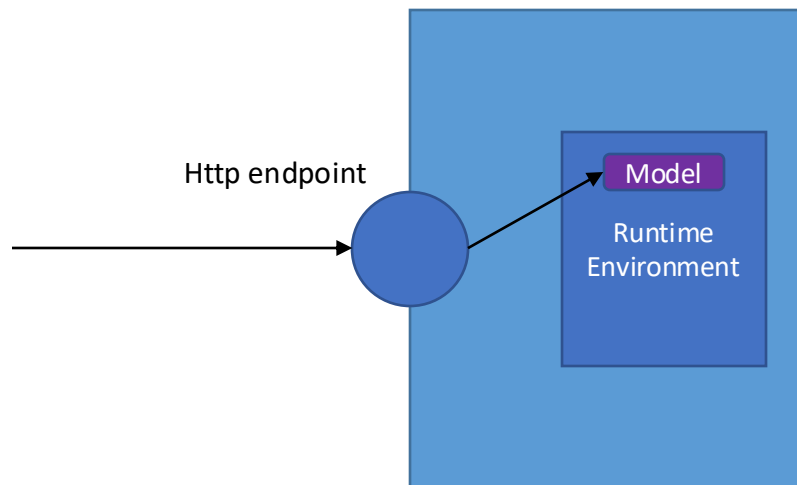
To fairness check

Categories of Model Deployment



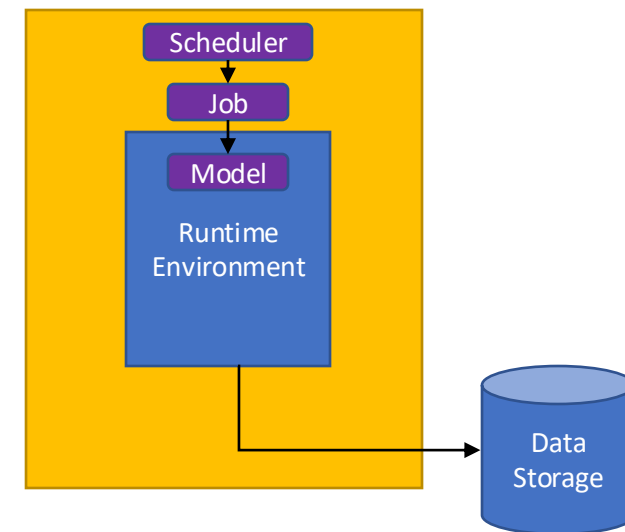
Real-time scoring

One or a 'small' number of records are scored



Batch scoring

Whole datasets are processed using a model, such as in daily scheduled jobs

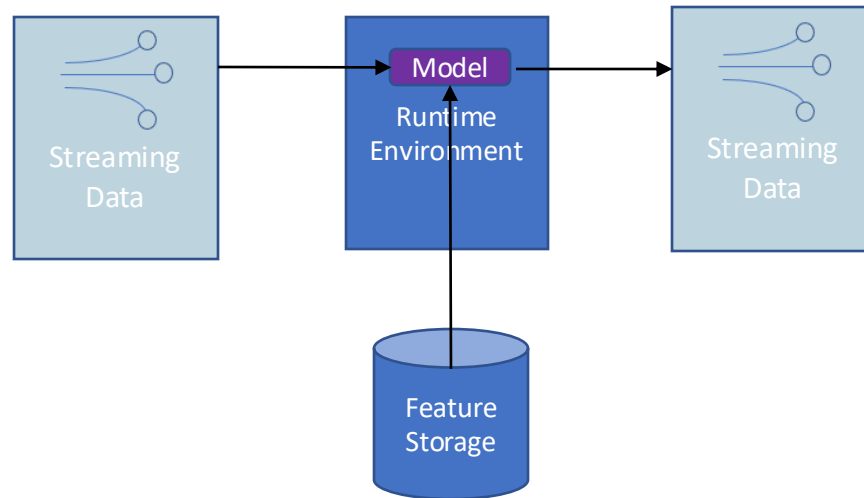


Categories of Model Deployment (cont.)



Stream scoring

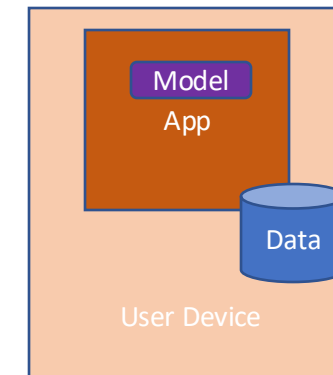
An application continuously applies inference on the data and returns it to another streaming



Real time Feature Store Serving API to retrieve additional Static Features for inference purposes

Edge scoring

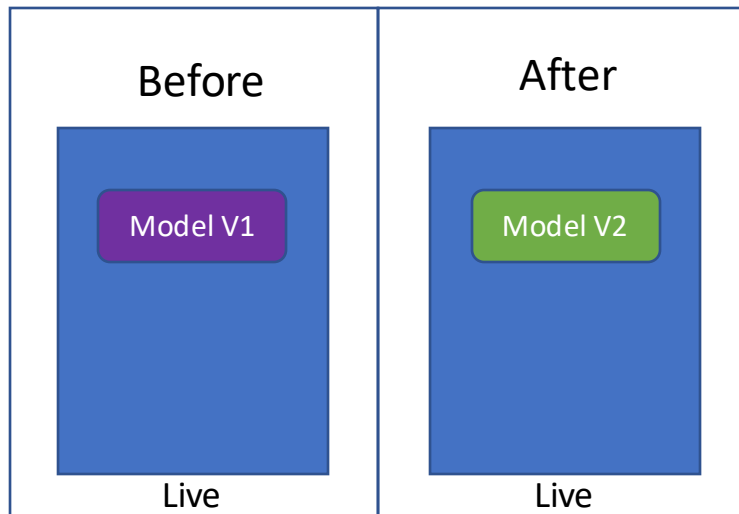
Trained model embedded in application that runs on user device or local server for smallest latency or disconnected scenarios



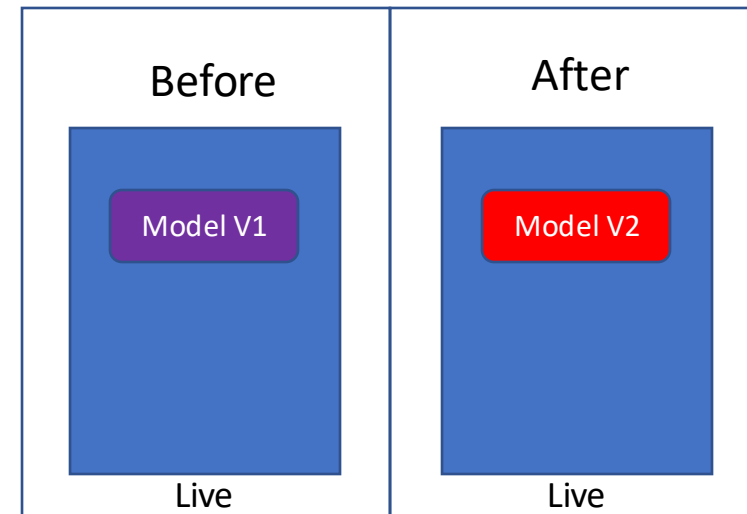
Data to score the model usually on the same device

Deployment Strategies – Basic Deployment

- All nodes within a target environment are updated at the same time with a new artifact version. If something goes wrong need to roll back to previous version.



Or

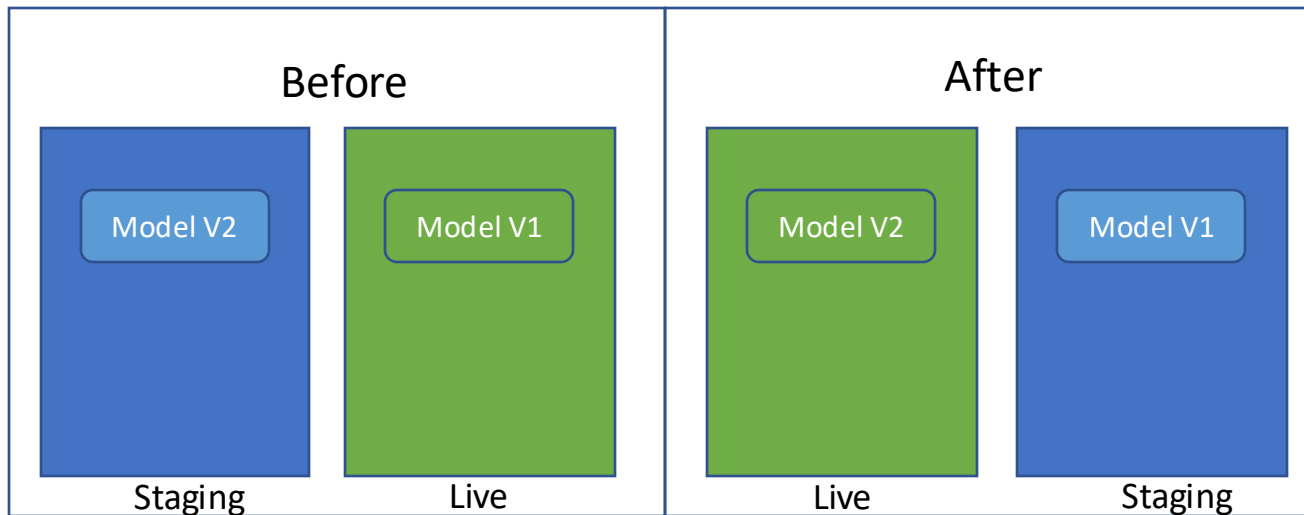


Something went wrong

- The benefits of this strategy are that it is simple, fast, and cheap. But it is **risky**.
- Use this strategy if 1) your application service is not business, mission, or revenue-critical, or 2) your deployment is to a lower environment, during off-hours, or with a service that is not in use.

Deployment Strategies – Blue-Green Deployment

- Blue-green deployment is a deployment strategy that utilizes **two identical environments**, a “blue” (aka staging) and a “green” (aka production) environment with different versions of an application or service.
- Quality assurance and user acceptance testing are typically done within the blue environment that hosts new versions or changes.
- User traffic is shifted from the green environment to the blue environment once new changes have been testing and accepted within the blue environment.



Notice they are different but identical environments

Pros

Simple, fast, well-understood, and easy to implement.

Rollback is also straightforward

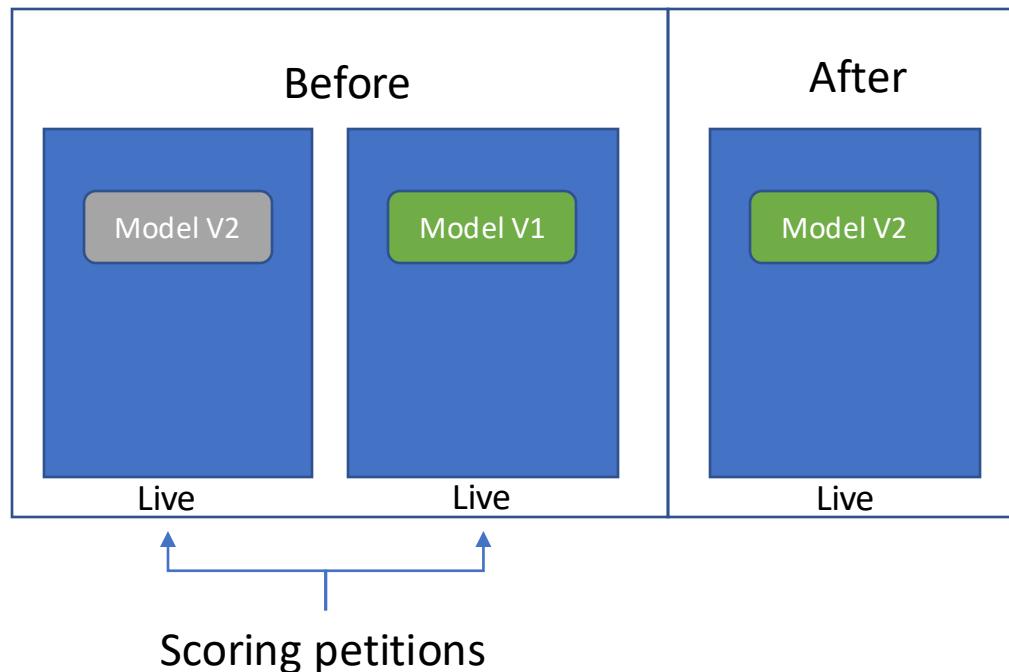
Cons

Creating testing data/inbound traffic is not always easy/cheap

Ensuring infrastructure is the same in both environments could also be a challenge

Deployment Strategies – Shadow Testing

- The new model version is deployed into the live environment alongside the existing model.
- All live scoring is handled by the incumbent model version, but each new request is then scored again by the new model version and the results logged, but not returned to the requestor.
- Once sufficient requests have been scored by both versions, the results can be compared statistically.



Pros

New model scores real data

Rollback to model V1 if there is a problem should be easy

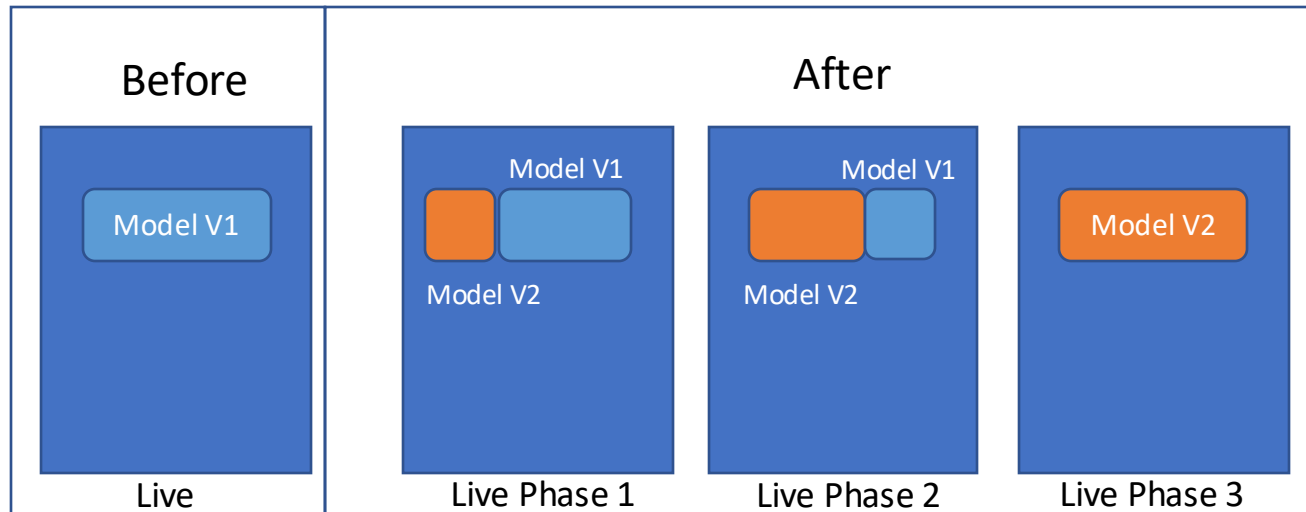
Cons

You need two 'exact' production environments working alongside

You are sending all the traffic to both models

Deployment Strategies – Canary Deployment (A/B Testing)

- A canary deployment is a deployment strategy that releases an application or service incrementally to a subset of users
- The stable version of the model is kept in production, but a certain percentage of the workload is redirected to the new model, and results are monitored
- A canary release is the lowest risk-prone, compared to all other deployment strategies, because of this control.



Pros

It's cheaper because it does not require two production environments

Cons

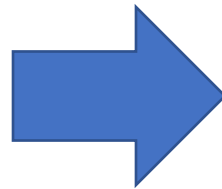
Although you test in production with real data it is a 'controlled' sample of the data until final rollout

Containerization

- Managing the versions of a model is much more than just saving its code into a version control system: it is necessary to provide an **exact description of the environment** (including, for example, all the Python libraries used as well as their versions, the system dependencies that need to be installed, etc.).
- But storing this metadata is not enough. Deploying to production should automatically and reliably rebuild this environment on the target machine.
 - The target machine will typically run multiple models simultaneously, and two models may have **incompatible dependency versions**.
 - Several models running on the same machine could **compete for resources**, and one misbehaving model could hurt the performance of multiple cohosted models.



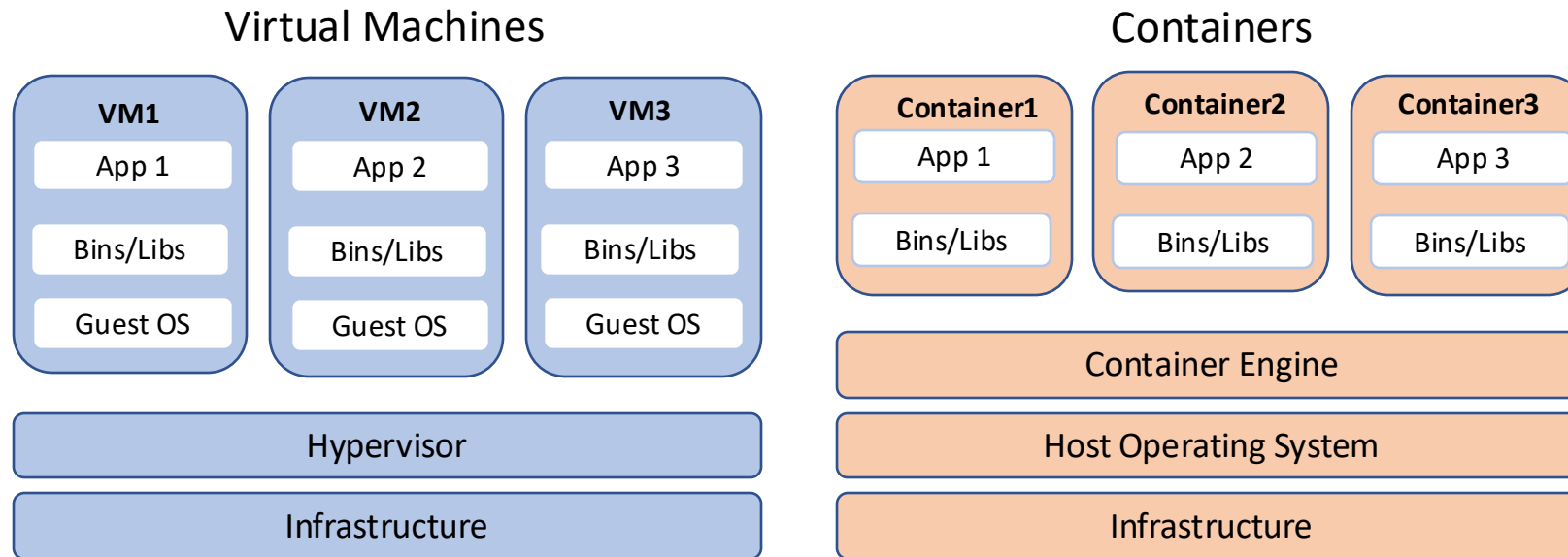
Containerization



- These tools bundle an application together with all of its related configuration files, libraries, and dependencies that are required for it to run across different operating environments.
- Unlike virtual machines (VMs), containers do not duplicate the complete operating system; multiple containers share a common operating system and are therefore far more resource efficient.

Virtual Machines vs. Containers

- Virtual Machines virtualize an entire machine and containers only software layers above the operating system



Providers: VMWare, VirtuaBox

Docker, LXC

Pros: Full standalone system -> more security
Can be interactively developed

High iteration speed
Containers repositories
Storage costs

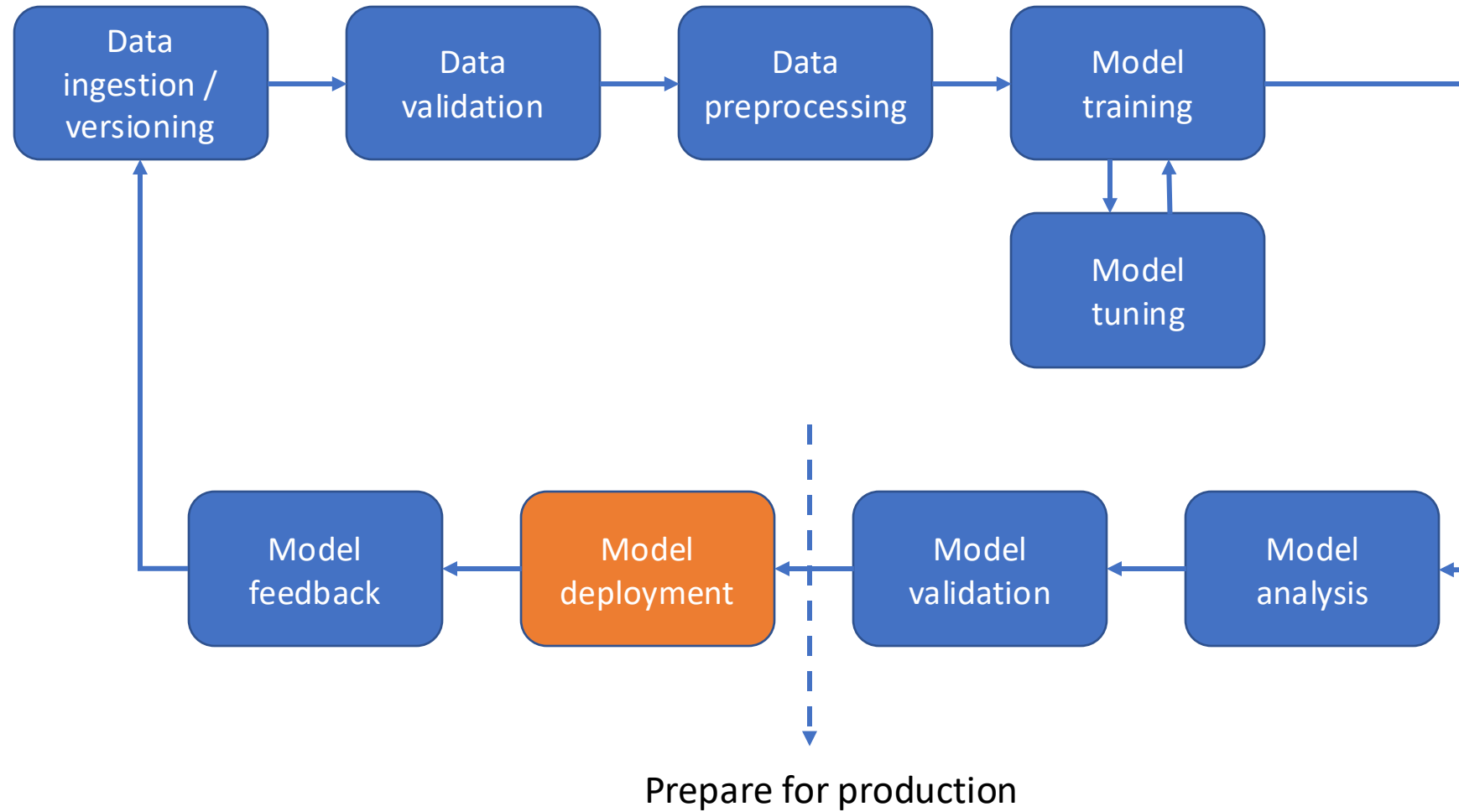
Cons: Slow iteration speed
Storage costs

Security concerns must be addressed

Docker

Scaling

Model Deployment



Scalability – The Problem

- Imagine reaching a point where we need to incorporate more machines and resources to handle the traffic coming into our model
- We need scalability to grow the capacity of our resources to be able to support a higher demand
- Many engineering teams don't pay the necessary attention to it.
- Main reason: we don't have a clear plan on how to scale things up from the beginning

We will discuss what's a typical process to handle a steady growth in the user base, and what tools and techniques one can incorporate from a machine learning perspective

We have trained a ML model and are ready to deploy...

First Decision

Set up our own
server?

Take care of the hardware
We need to solve all the scalability
things as we grow

Deploy in a Cloud
Provider?

Google Cloud, AWS, Azure, IBM Cloud...

We take advantage of all the
ready-to-use infrastructure

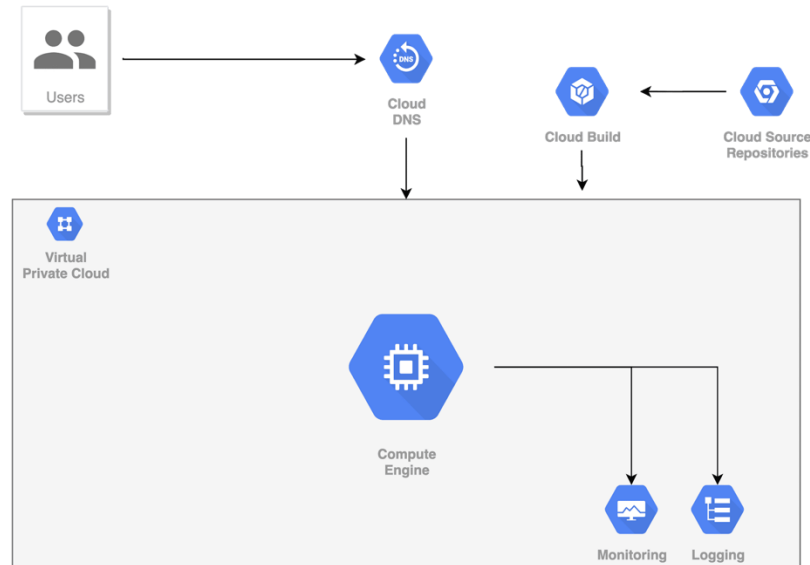
Let's follow a deployment story...

Vertical vs horizontal scaling

- **Vertical scaling (scaling-up)**

we add more power (CPU, Memory, GPU) to an existing machine to handle the traffic.

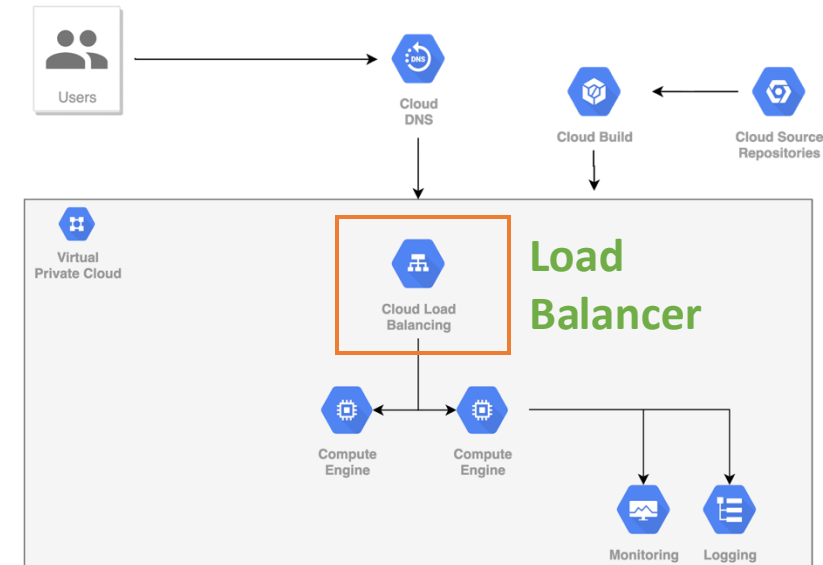
- Our first reaction is to scale vertically.
- What happens if we keep doing that?
- We're starting to hit a limit and realize that we can't keep scaling up -> **we have to scale out.**



- **Horizontal scaling (scaling out)**

we add more machines to the network and share the processing and memory workload across them.

- Create a new compute engine instance
- Then replicate the application and have them both run and serve traffic simultaneously.
- We need a **load balancer**
- Then check if neither instance is struggling and their response times go back down.



Autoscaling to solve random spikes

- Autoscaling is a method used in cloud computing that alters the number of computational resources based on the load.
- Typically, this means that the number of instances goes up or down automatically, based on some metrics.
- Usually, autoscaling can have two flavors: scheduled scaling and dynamic scaling.

Scheduled Scaling

- If we know in advance that the traffic will vary in a specific period of time.
- Manually instruct the system to create more instances

Dynamic Scaling

- Monitors some predefined metrics and scales the instances when they surpass a limit (CPU or Memory utilization, requests count, response time, etc.)
- Example: Number of instances double when number of requests is more than 100. When requests fall again, the instances will go back to normal number of instances

Kubernetes



- Kubernetes, also known as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications.
- It groups containers that make up an application into logical units for easy management and discovery.
- Kubernetes builds upon 15 years of experience of running production workloads at Google, combined with best-of-breed ideas and practices from the community.

- Process automation
- Load balancing
- Self-monitoring
- Storage orchestration

OpenShift



- Red Hat's OpenShift is a cloud-based container platform that functions as both a Platform-as-a-Service and Container Orchestration Engine.
- At its core, OpenShift is an open-source tool that leverages the Kubernetes platform to manage Docker containers
 - Workload management
 - Self-monitoring
 - Centralized policy provisioning
- With OpenShift, developers can deploy containerized applications in an Integrated Development Environment (IDE), while employing Kubernetes to manage them.

Why Kubernetes if we have Docker?

- Scenario: run multiple docker containers on multiple machines to support an enterprise level ML application with varied workloads during day and night
- Can we do it manually? -> It is a lot of work to do it manually
- Start the right containers at the right time, figure out how they can talk to each other, handle storage considerations and deal with failed containers or hardware
- Kubernetes solve this problem allowing large number of containers to work together in harmony reducing the operational burden



Docker and Kubernetes are two different technologies:

- Docker allows to containerize applications
- Kubernetes is a container management system that allows to create, scale and monitor hundreds and thousands of containers

Scalable and Elastic Systems

Scalable System

- A computational system is said to be **horizontally scalable** (or just scalable) if it is possible to incrementally **add more computers** to expand its processing power.
- For example, a Kubernetes cluster can be expanded to hundreds of machines.
- However, if a system includes only one machine, it may be challenging to incrementally upgrade it significantly, and at some point, a migration to a bigger machine or a horizontally scalable system will be required (and may be very expensive and require interruption of service).

Elastic System

- An **elastic system** allows, in addition to being scalable, **easy** addition and removal of resources to match the compute requirements.
- For example, a Kubernetes cluster in the cloud can have an auto-scaling capability that automatically adds machines when the cluster usage metrics are high and removes them when they are low.
- In principle, elastic systems can optimize the usage of resources as they automatically adapt to an increase in usage without the need to permanently provision resources that are rarely required.

Deploying Docker in GCP